

CSE 331: Computer Security Fundamentals

Spring 2026

Web Security:
Browser Security Model

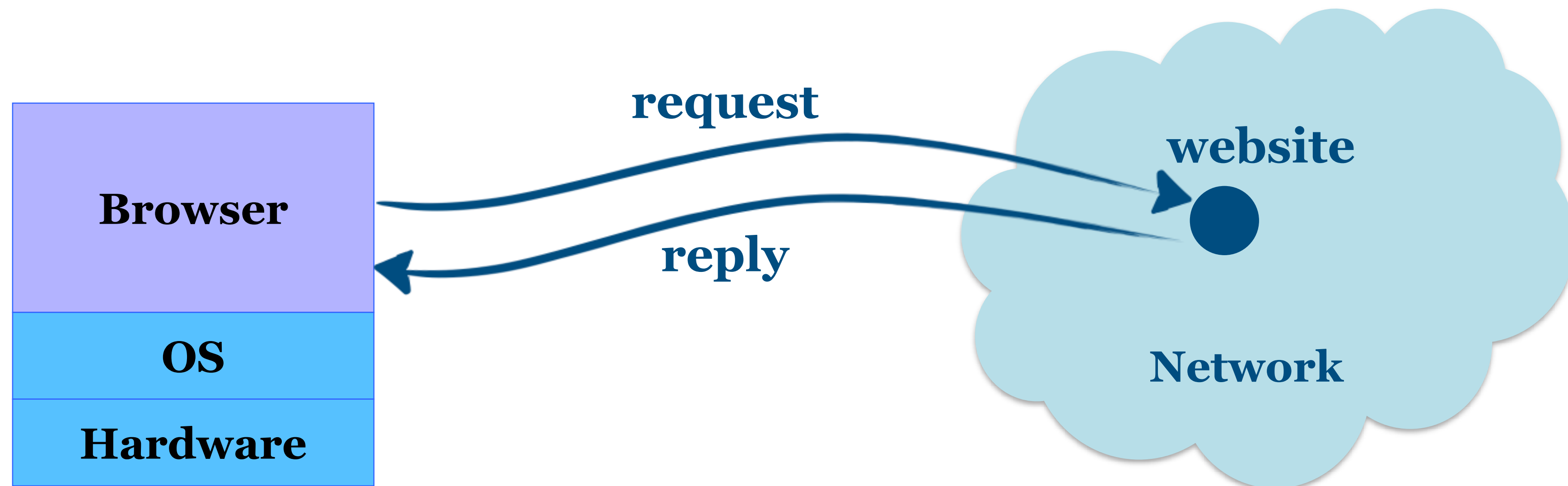
Amir Rahmati
Xinrui Yu



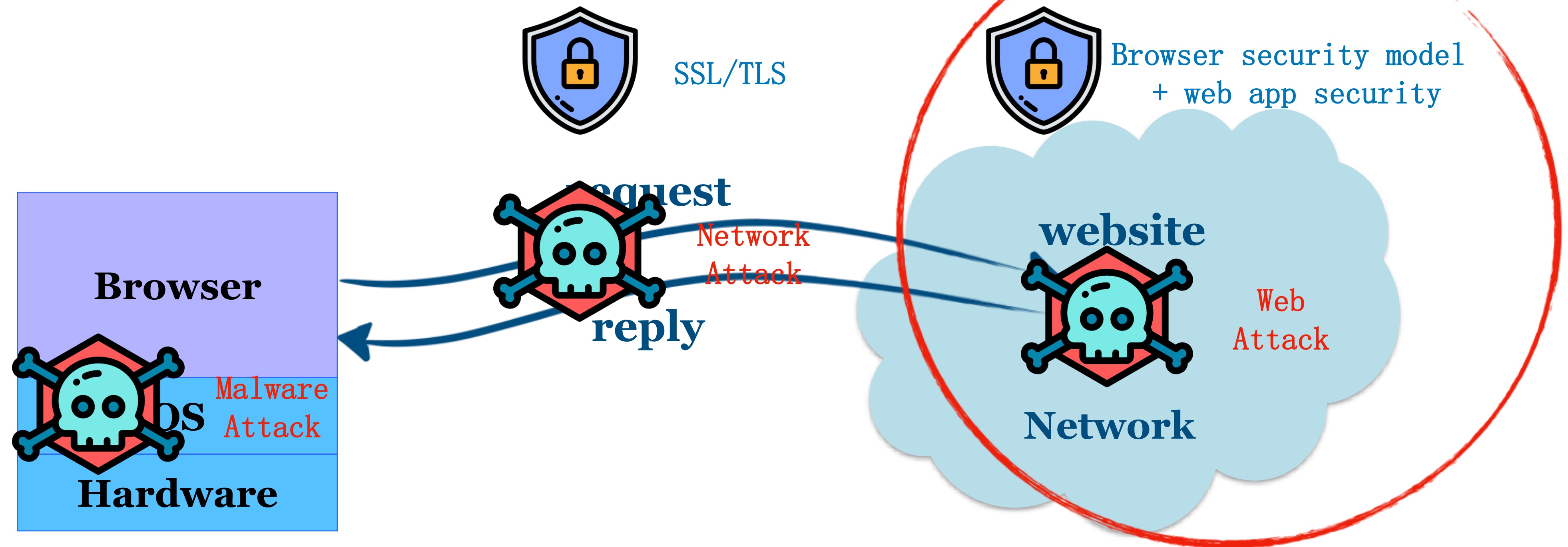
Stony Brook Institute
at Anhui University

安徽大学纽约石溪学院

Big Picture: Browser and Network



Where Does the Attacker Live?



Threat Model

- Controls a malicious website (**attacker.com**)
 - Can even obtain SSL/TLS certificate for site
- User visits attacker.com – why?
 - Phishing email, enticing content, search results, placed by an ad network, blind luck ...
- Attacker has no other access to user machine!
- Variation: good site **honest.com**, but:
 - An iframe with malicious content included
 - Website has been compromised



Two Sides of Web Security

(1) Web browser

- Responsible for securely confining content presented by visited websites

(2) Web applications

- Online merchants, banks, blogging platforms, Google Apps ...
- Mix of server-side and client-side code
 - Server-side code written in Python, Java, PHP, etc.
 - Client-side code written in HTML, CSS, JavaScript
- Many potential bugs: XSS, XSRF, SQL injection

All of These Should Be Safe

- Safe to visit the attacker website
- Safe to visit two pages at the same time
- Safe delegation



Browser Security Model

Goal 1: Protect local system from web attacker

→ **Browser Sandbox**



Goal 2: Protect/isolate web content from other web content

→ **Same Origin Policy (+ sandbox)**



Browser Are Like Virtual Machines

Feature	JVM	Browser
Platform Independence	Runs Java bytecode on any OS/hardware	Runs web apps (HTML/CSS/JS) on any device
Sandboxing	Isolates code from host system	Isolates each origin in a security sandbox
Security model	Bytecode verification, security manager	Same-origin policy, CSP, permissions

Browser Sandbox



Goals: Protect local system from web attacker; protect websites from each other

- E.g., safely execute JavaScript provided by a website
- No direct file access, limited access to OS, network, browser data, content from other websites
- Tabs and **iframes** in their own processes
- Implementation is browser and OS specific

- For example, see: https://chromium.googlesource.com/chromium/src/+/_/master/docs/design/sandbox.md

From Chrome Bug Bounty Program

	High-quality report with functional exploit	High-quality report	Baseline
Sandbox escape / Memory corruption in a non-sandboxed process	\$30,000	\$20,000	Up to \$15,000

Same Origin Policy

Goal: Protect/isolate web content from other web content

Website origin = (protocol, domain, port)

`http://www.example.com/dir/page.html`

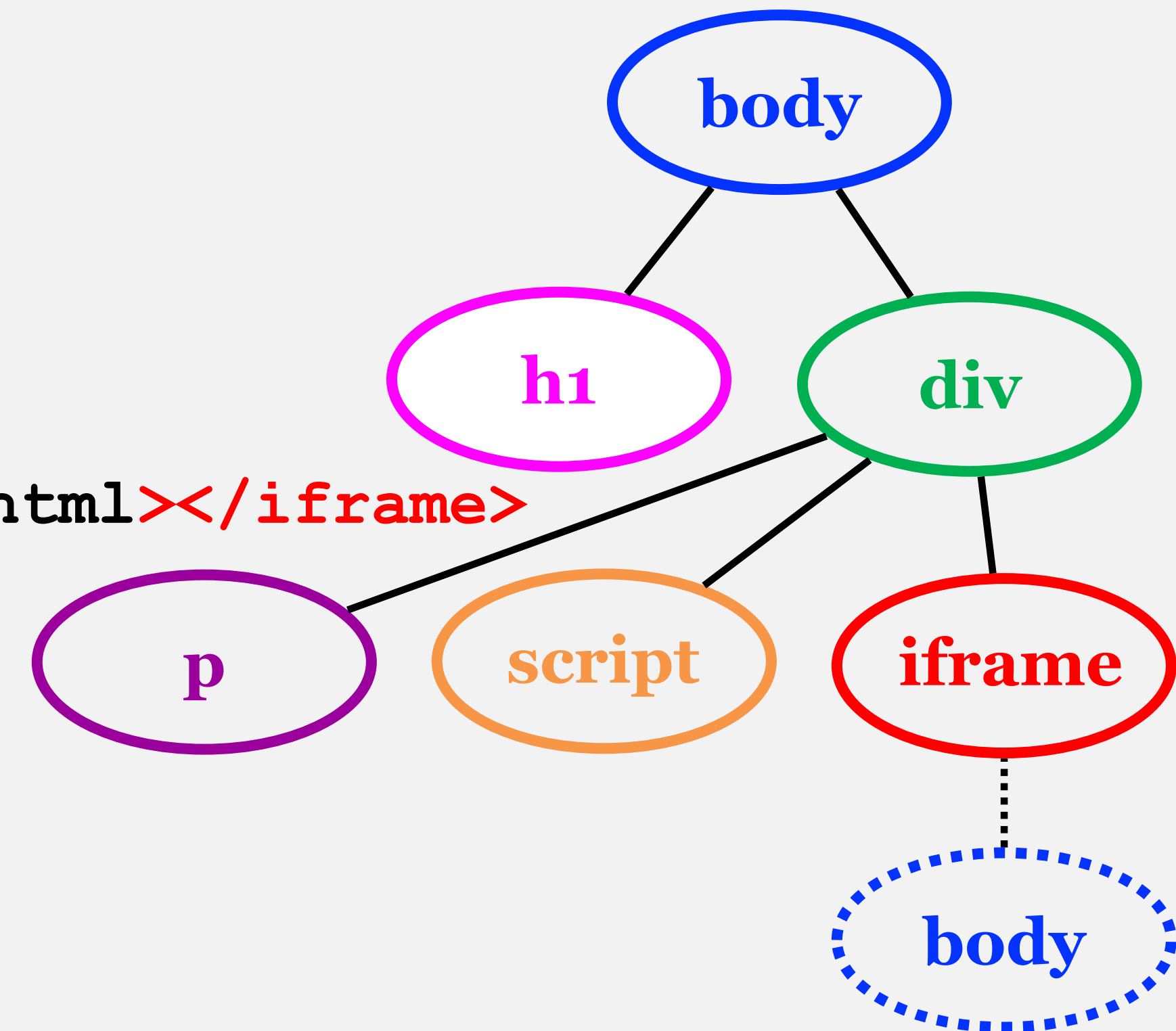
Compared URL	Outcome	Reason
<code>http://www.example.com/dir/page.html</code>		
<code>http://www.example.com/dir2/other.html</code>		
<code>http://www.example.com:81/dir/other.html</code>		
<code>https://www.example.com/dir/other.html</code>		
<code>http://en.example.com/dir/other.html</code>		
<code>http://example.com/dir/other.html</code>		
<code>http://v2.www.example.com/dir/other.html</code>		

[Example from Wikipedia]

HTML + Document Object Model

```
<html>
  <body>
    <h1>This is the title</h1>
    <div>
      <p>This is a sample page.</p>
      <script>alert("Hello world");</script>
      <iframe src=https://www.example.com/dir/page.html></iframe>
    </div>
  </body>
</html>
```

Document Object
Model (DOM)

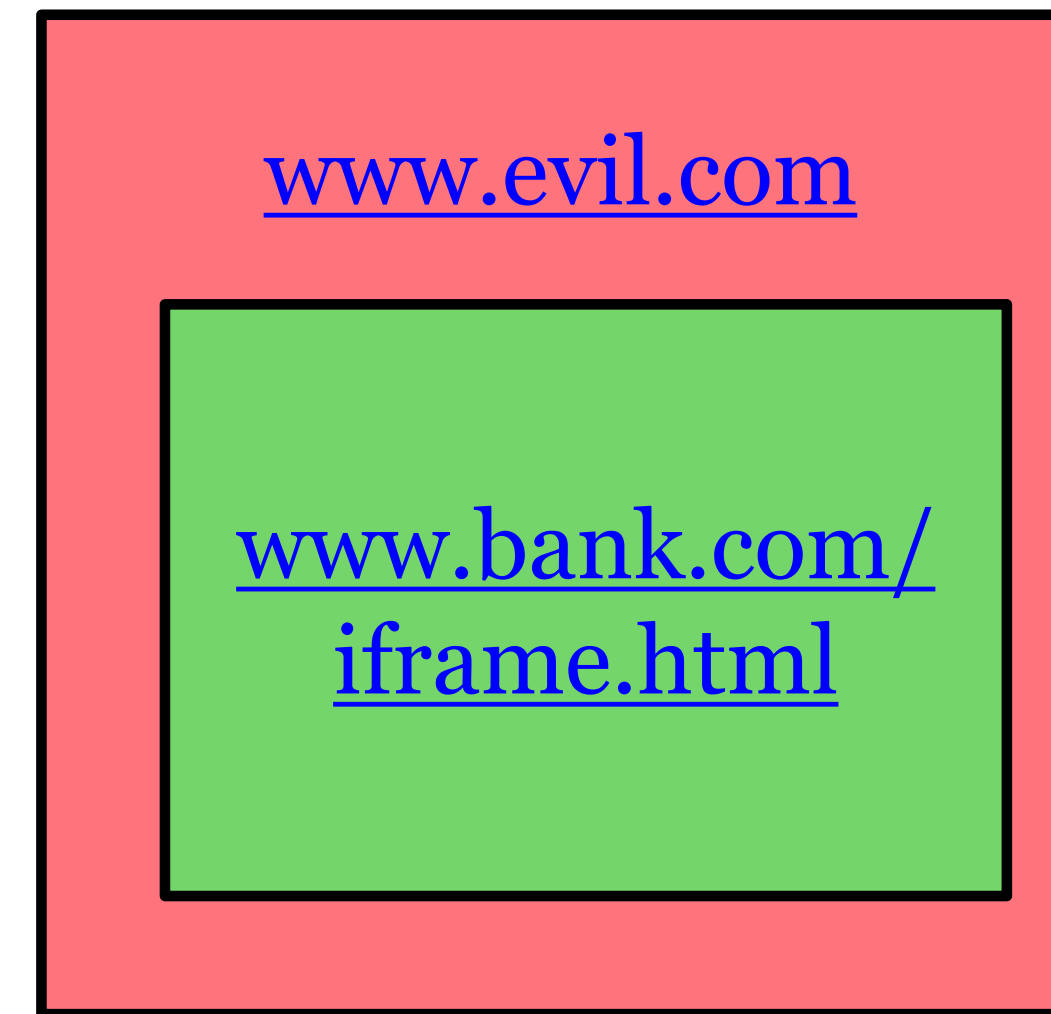


Same-Origin Policy: DOM

Only code from same origin can **access HTML elements** on another site (or in an iframe).



www.bank.com (the parent)
can access HTML elements
in the iframe
(and vice versa).

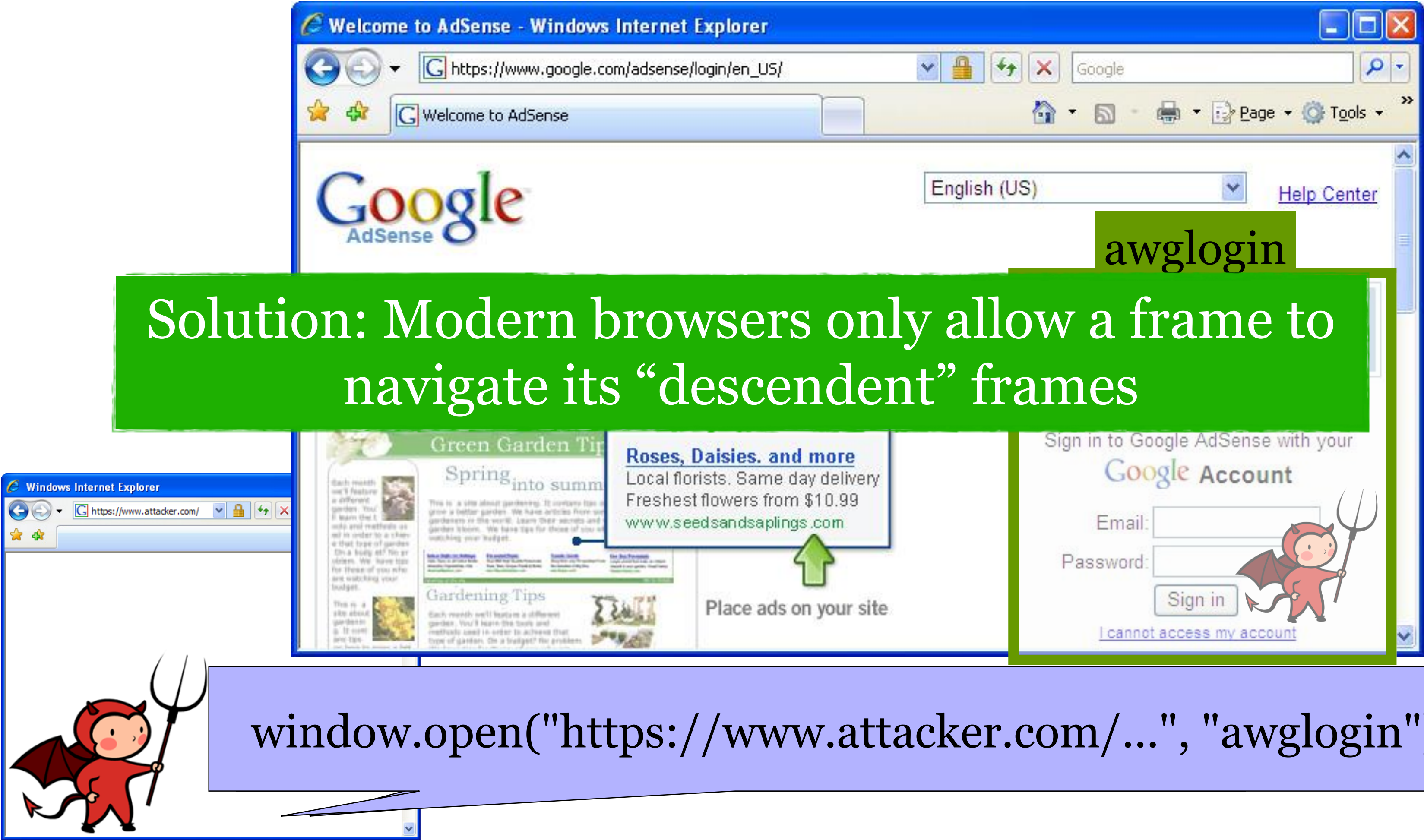


www.evil.com (the parent)
cannot access HTML
elements in the iframe
(and vice versa).

Question: Who Can Navigate a Frame?

awglogin

Solution: Modern browsers only allow a frame to navigate its “descendent” frames



```
window.open("https://www.attacker.com/...", "awglogin")
```

If bad frame can **navigate** sibling frames, attacker gets password!

Browser Cookies

- HTTP is stateless protocol
- **Browser cookies used to introduce state**
 - Websites can store small amount of info in browser
 - Used for authentication, personalization, tracking...
 - **Cookies are often secrets**



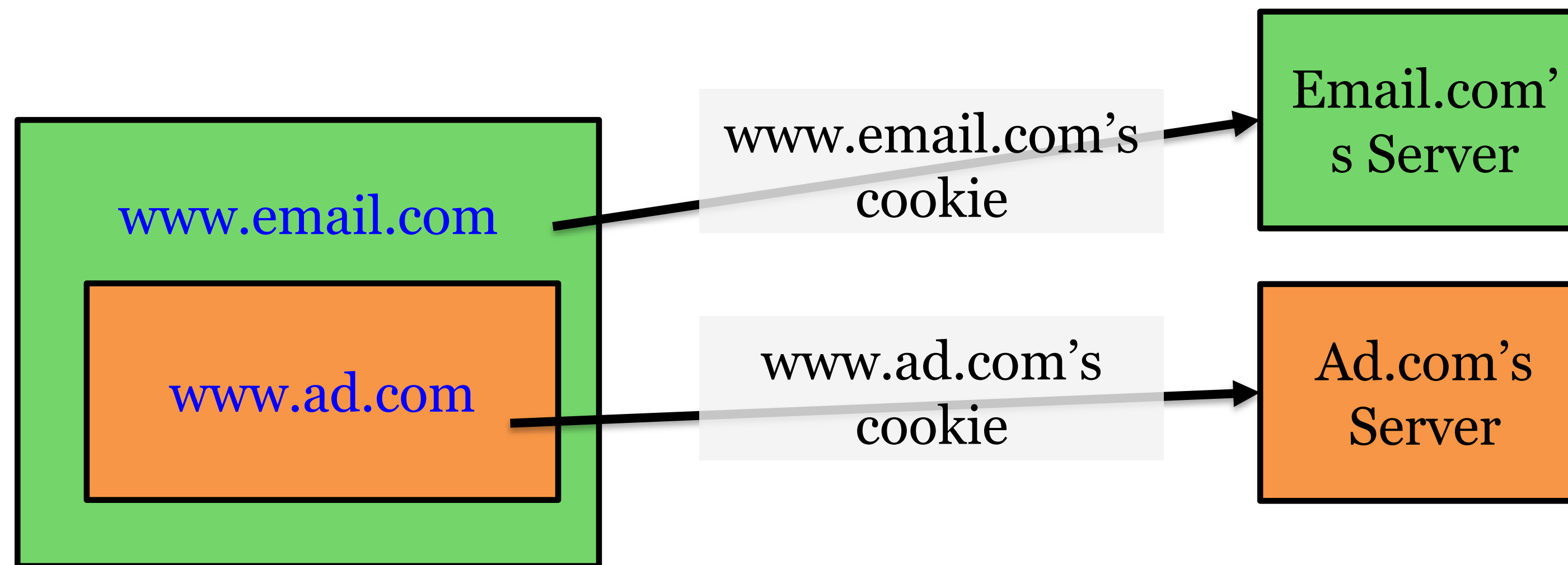
Cookies and HTTP(S)

- Cookies are just **plain-text headers in HTTP** messages
- If you use HTTP only:
 - Anyone can see what those cookies are!
 - E.g., WiFi in coffee shop attacker
 - FireSheep browser plugin
- One of the big reasons why the big push towards HTTPS



Same Origin Policy: Cookie Reading

- Websites can only read/receive cookies from the same domain
 - Can't steal login token for another site!



Same Origin Policy: Cookie Writing

Which cookies can be set by **login.site.com**?

allowed domains

login.site.com

.site.com

(**user.site.com**,
game.site.com,
ad.site.com, etc.)

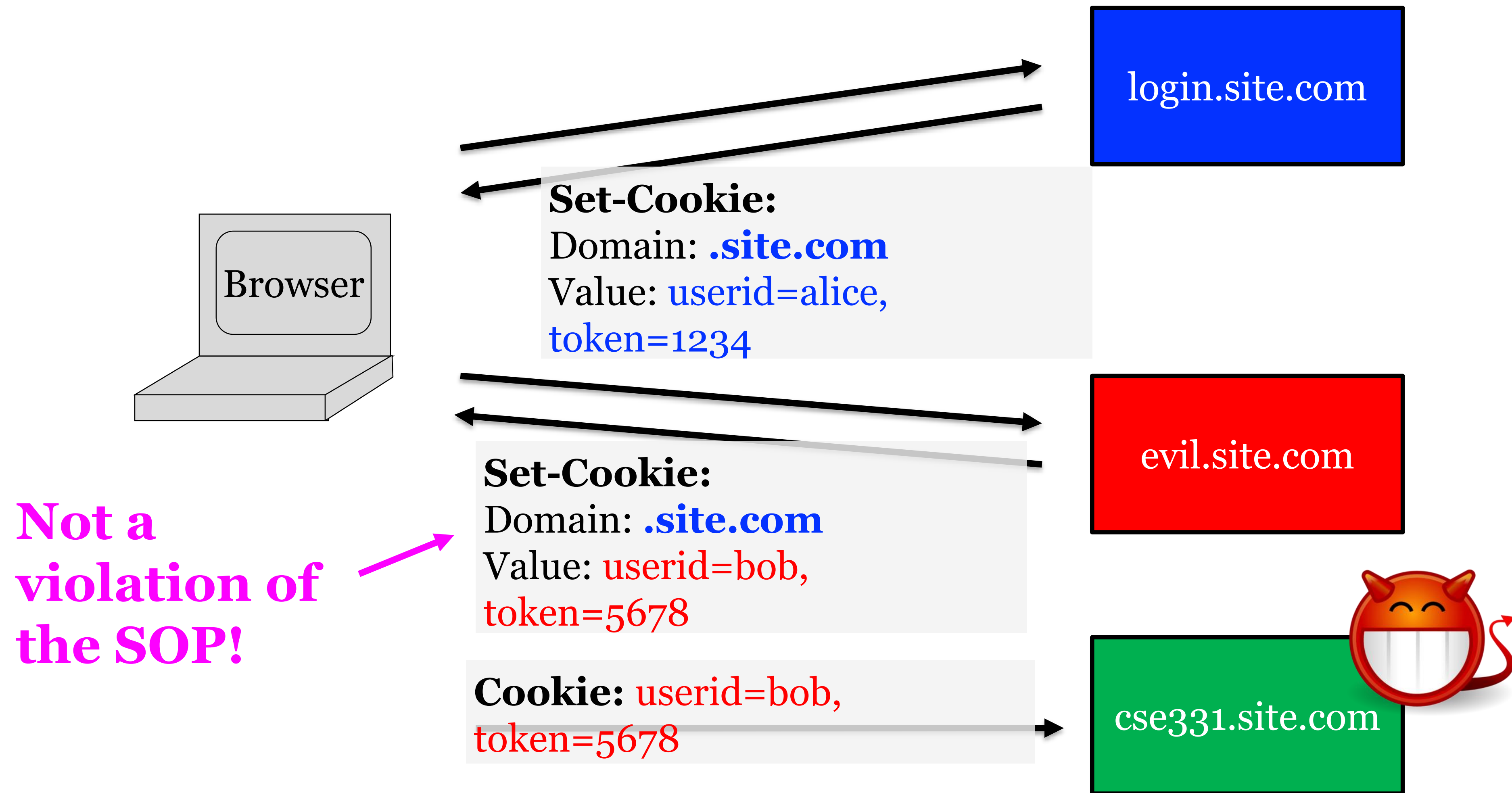
disallowed domains

othersite.com

.com

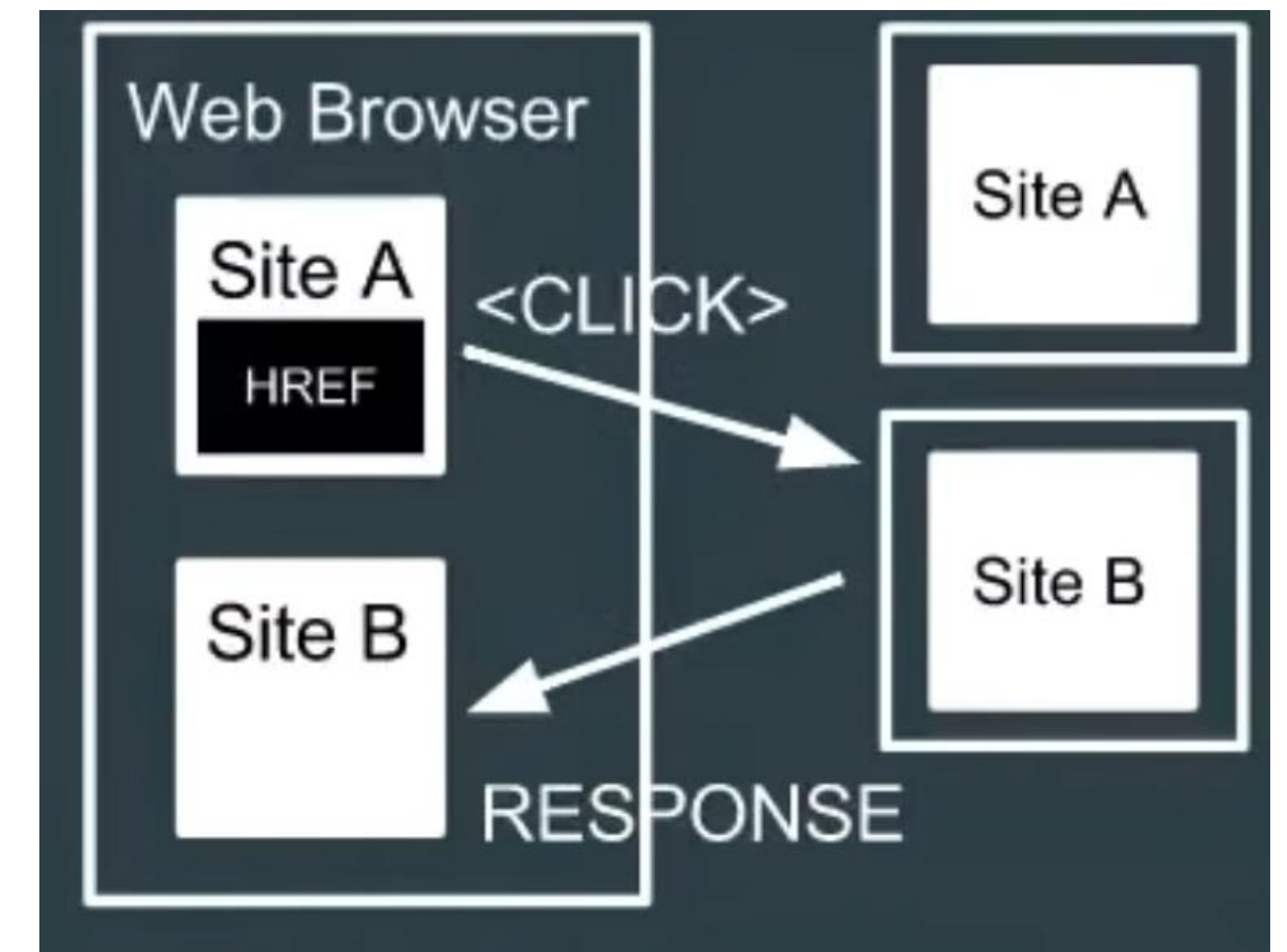
login.site.com can set cookies for all of **.site.com (domain suffix)**,
but not for another site or top-level domain (TLD)

Problem: Who Set the Cookie?



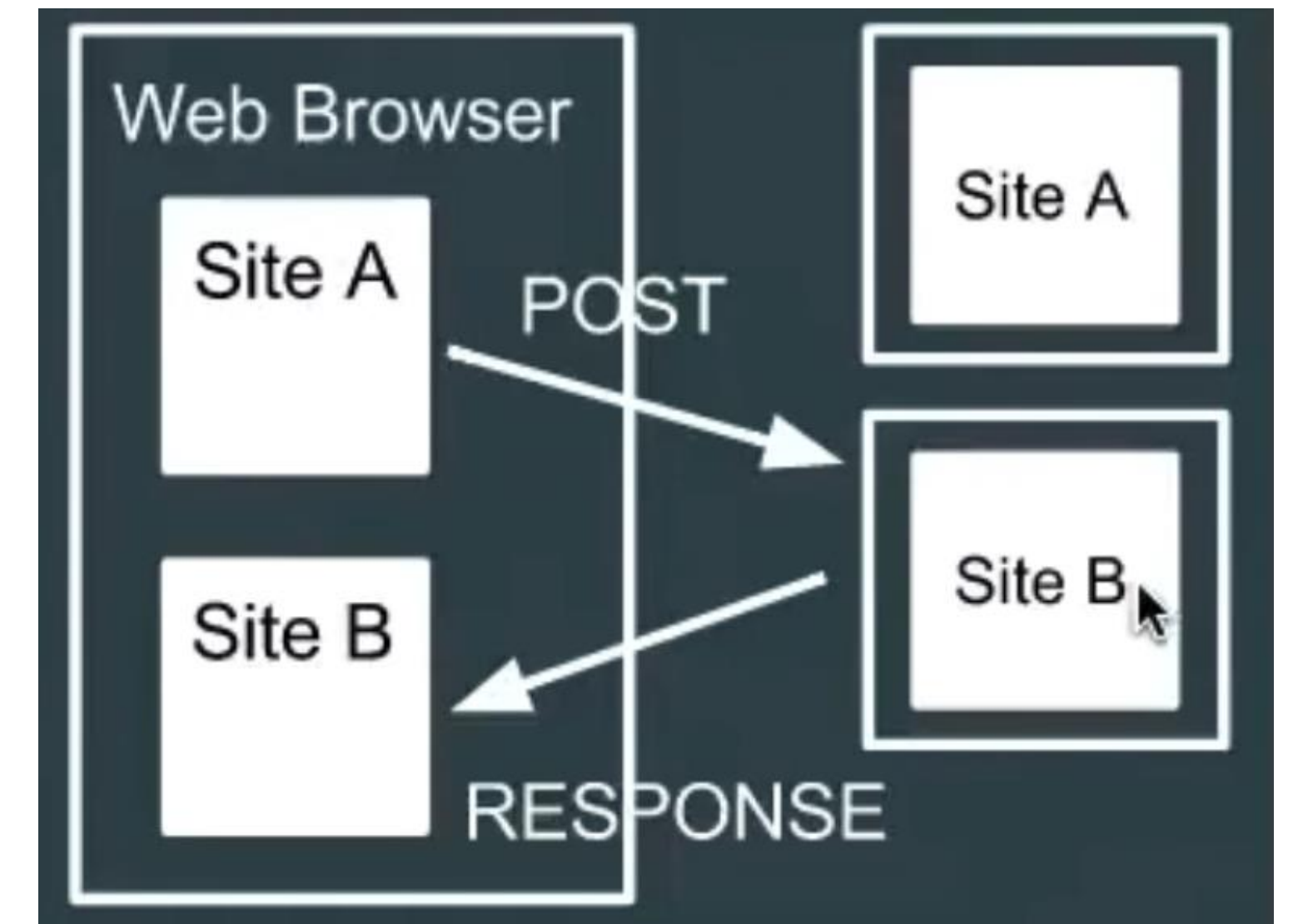
Same-Origin Policy: Links

- Site can hyperlink to any other site
- **Response loads in a new context**
 - Originating site is replaced by the accessed site
- **Any site can link, but can't read the response**



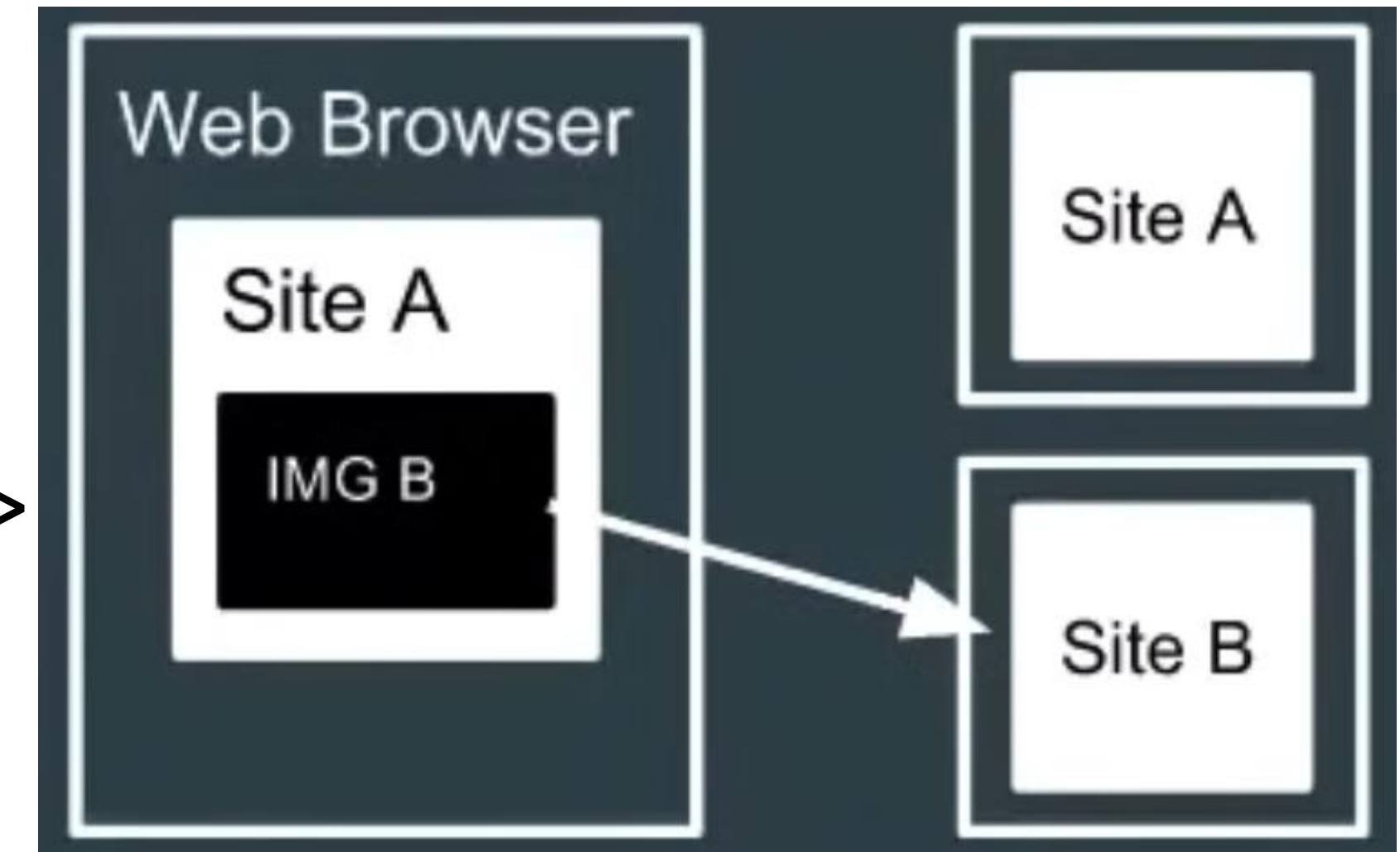
Same-Origin Policy: Forms

- Forms can POST to any site
 - `<form action="http://website.com" method="POST">`
- Response loads in a new context
 - Originating site is replaced by the accessed site
- Any site can POST, but can't read the response



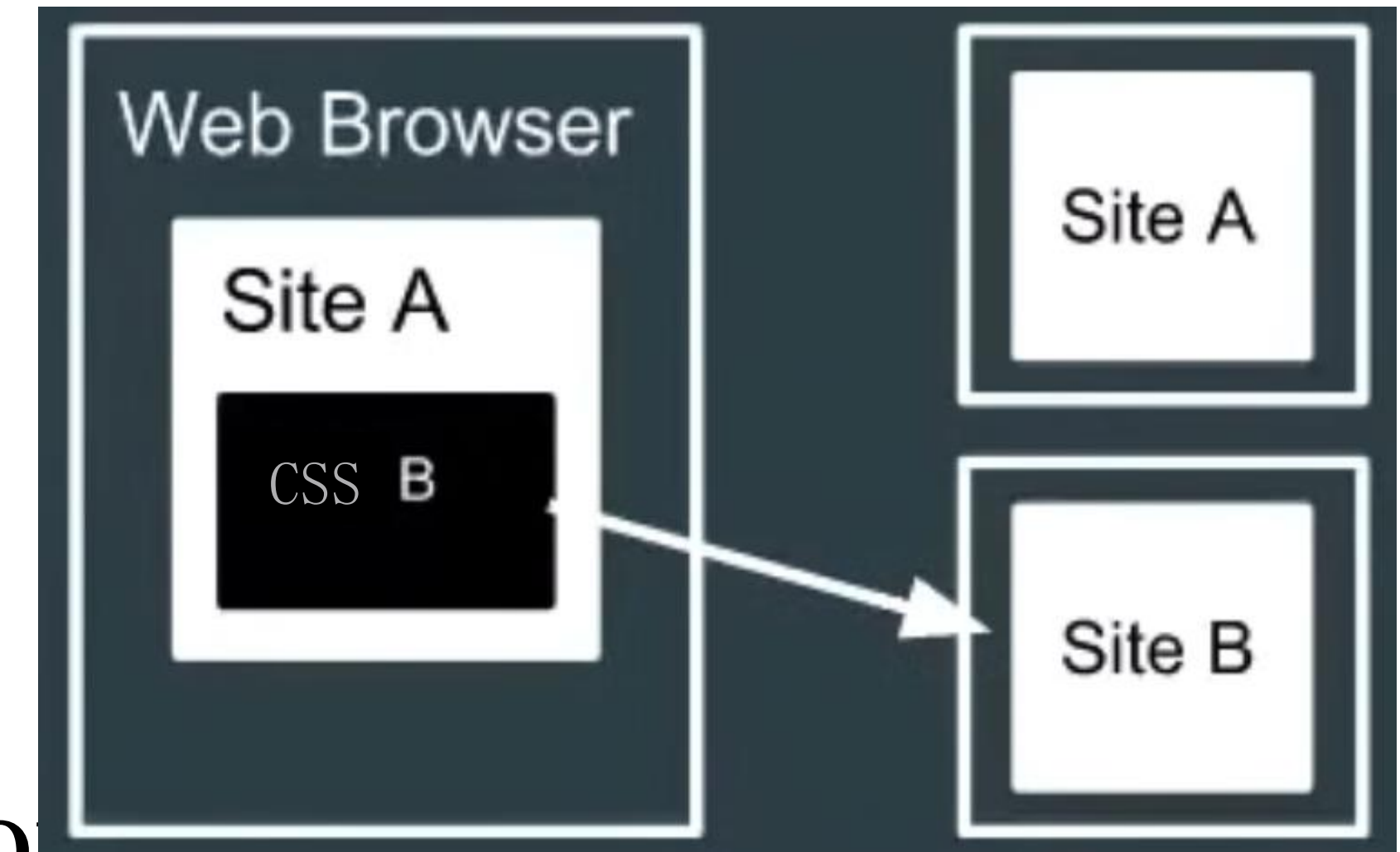
Same-Origin Policy: Images

- Images can POST to any site
 - ``
- Can't read the image data from the other sites.
- Same Story for other static media (movies, audios)



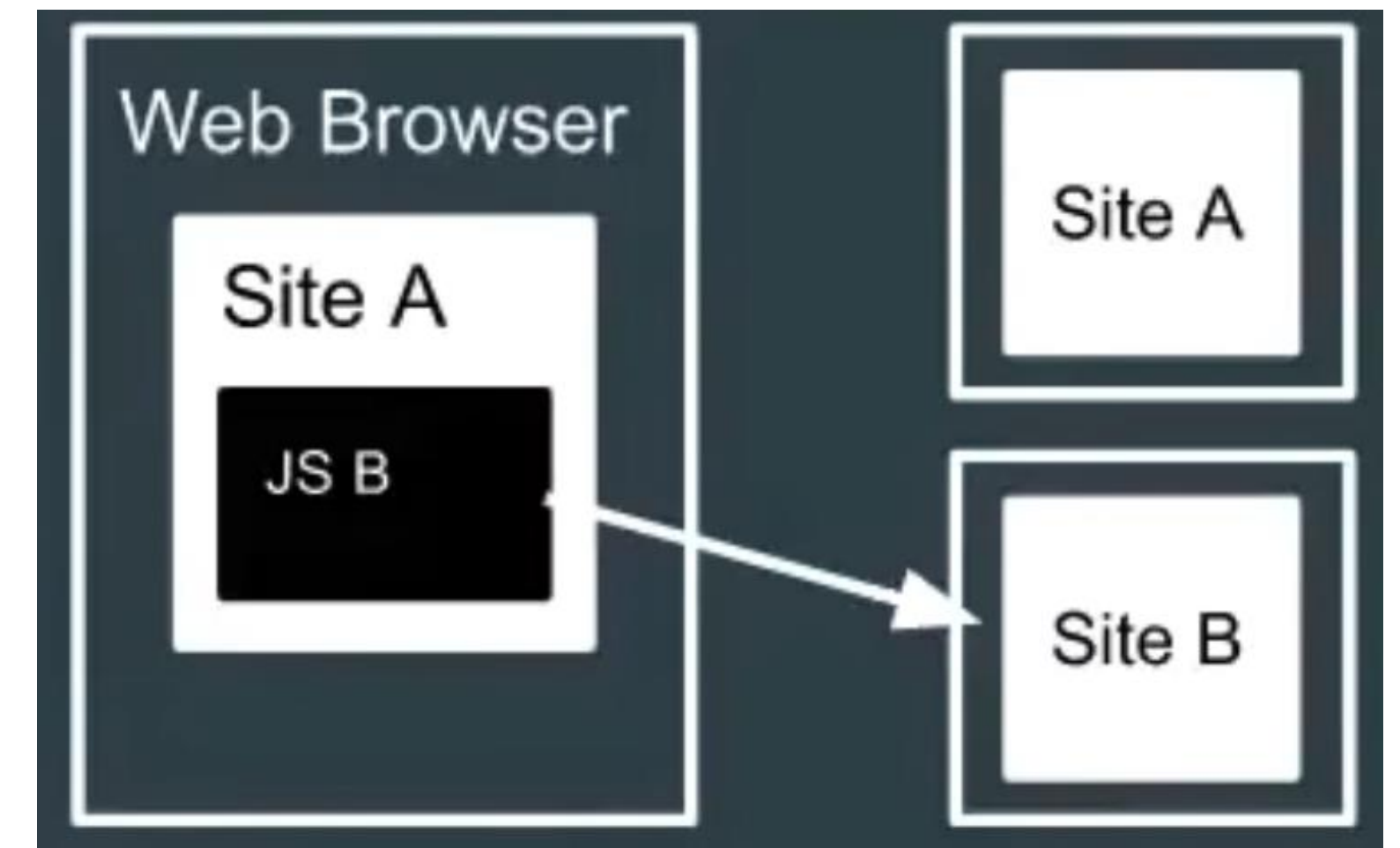
Same-Origin Policy: CSS

- Cascading Style Sheets
- Site can include CSS from another site
 - `<link rel="stylesheet" type="text/css" href="http://website.com/a.css"/>`
- Can't read the CSS data from other sites



Same-Origin Policy: js includes

- Forms can include javascript from other sites
 - `<script src="http://website.com/a.js" />`
- Javascript runs in the originating (embedding) site
- The code from Site B can access HTML elements and cookies on Site A
- If code in script sets cookie, under what origin will it be set? –site A

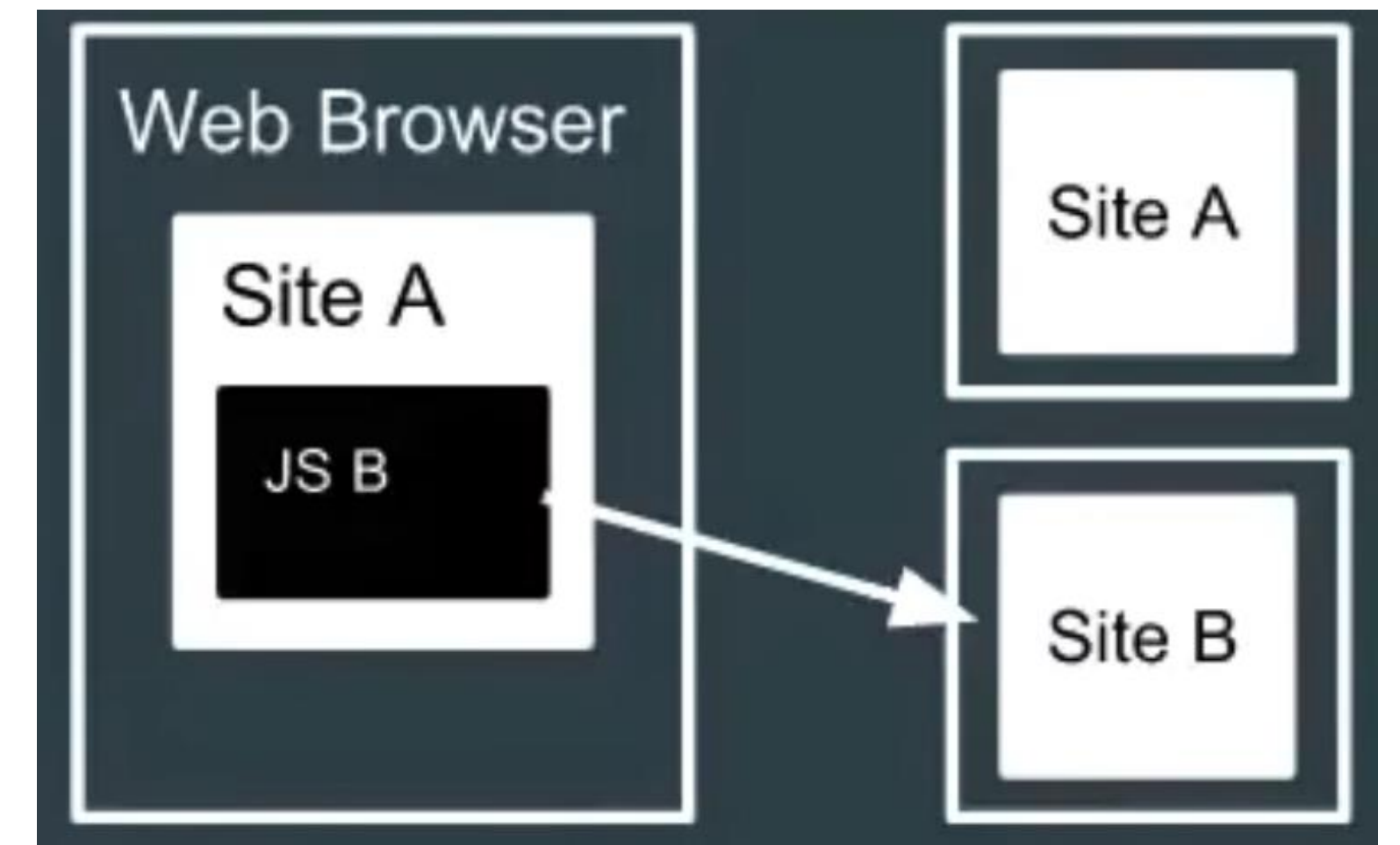


Early Internet legacy; mostly fixed today

Same-Origin Policy: JSONP

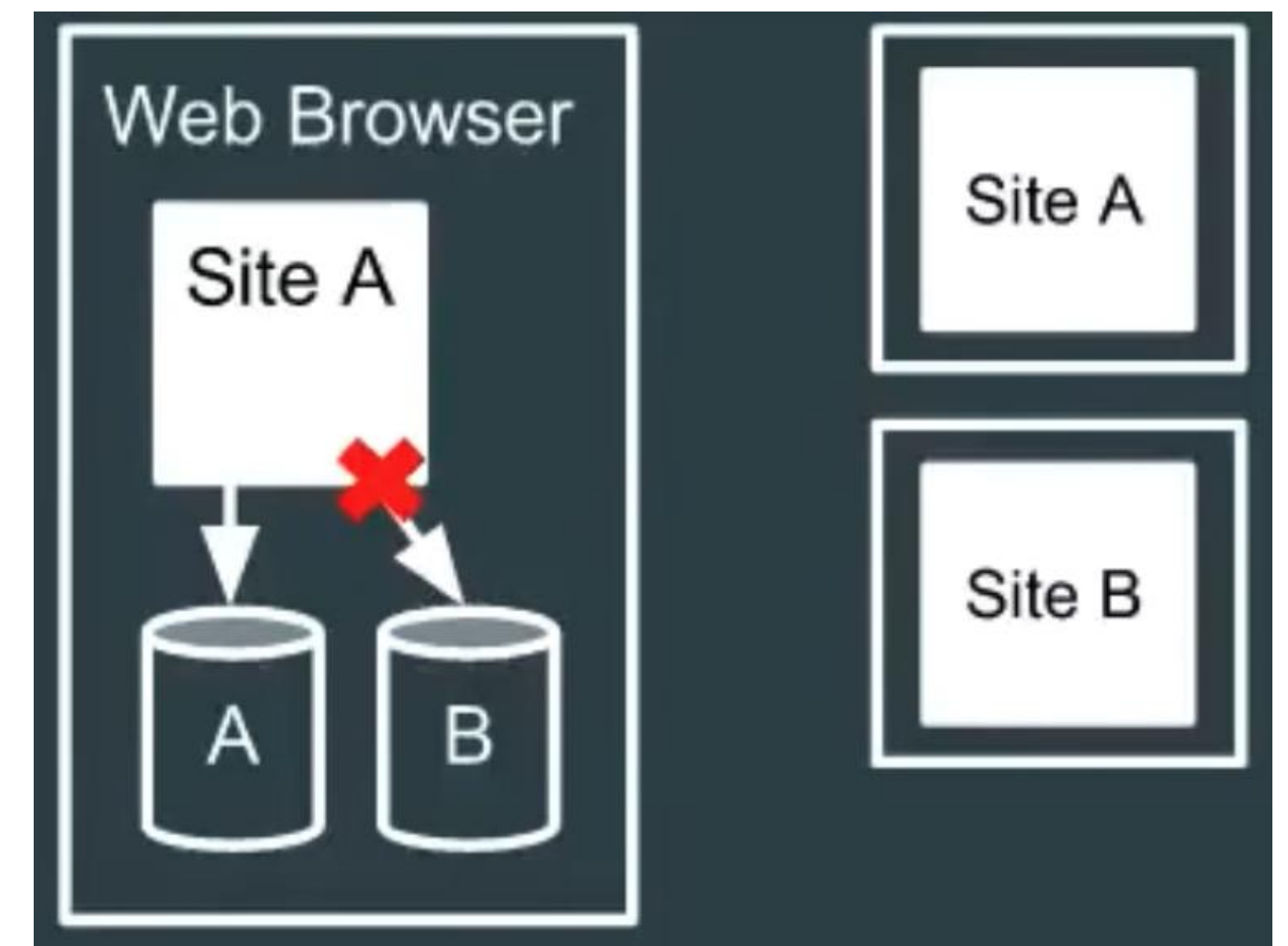
- JSONP abuses javascript to load data cross-origin

```
<script>function process (data) {...}</script>  
<script src="http://siteb.com/getdata.js" />  
=> process ({ "Name": "Foo", "Id": 1234,  
"Rank": 7 });
```



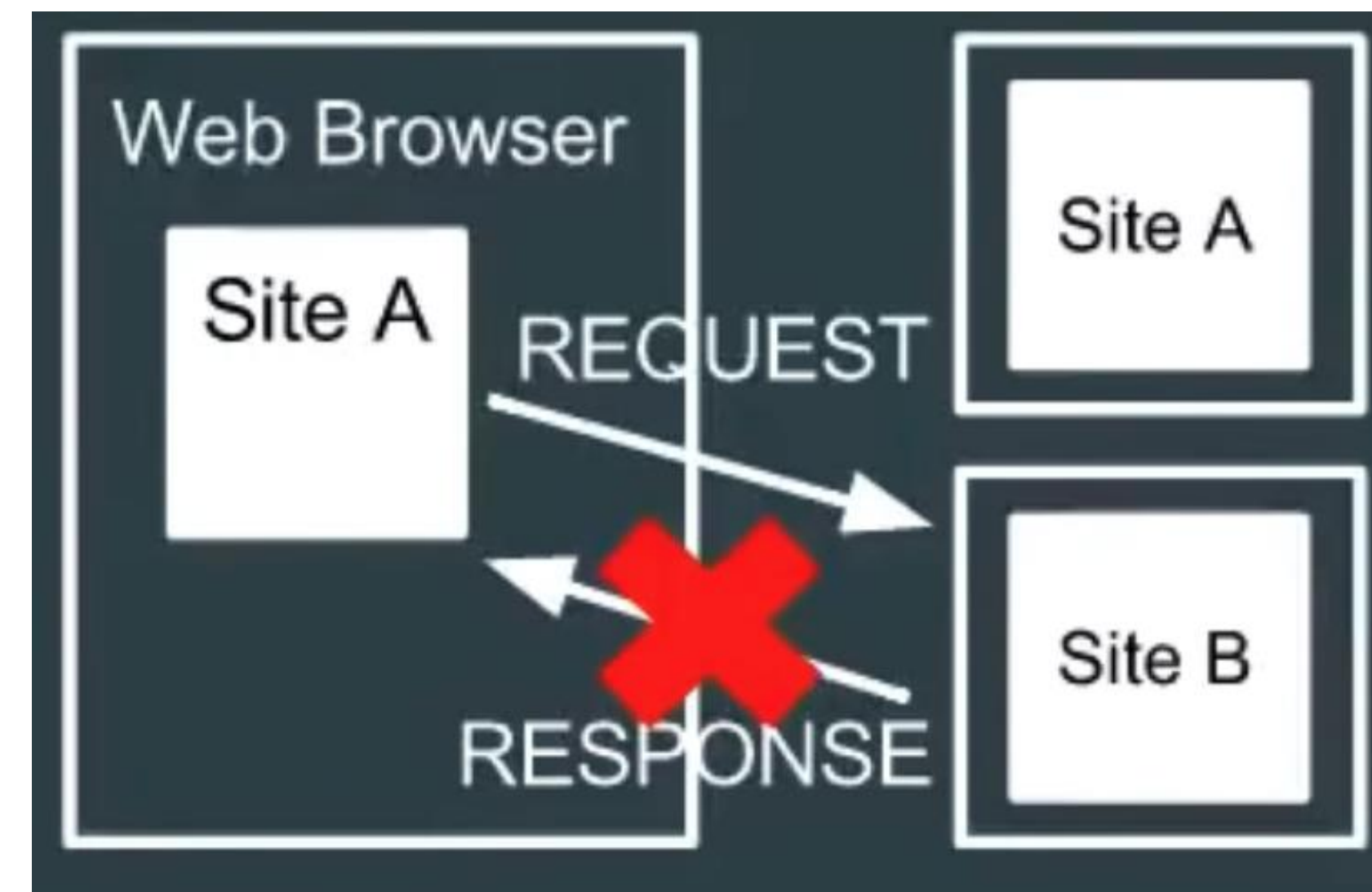
Same-Origin Policy: web storage

- Origin checks apply
- **Local Storage**
 - shared between windows with same origin
 - survives when window is closed
- **Session Storage**
 - only the current window
 - limited to lifetime of the window



Same-Origin Policy: XMLHttpRequest

- Site can GET or POST to any site
- **Browser will attach cookies or auth to the request**
- Only same-site requests can read responses



What about Browser Plugins?

- **Examples:** Flash, Silverlight, Java, PDF reader
 - Goal: enable functionality that requires transcending the browser sandbox
- **Increases browser's attack surface**

Java and Flash both vulnerable—again—to new 0-day attacks

Java bug is actively exploited. Flash flaws will likely be targeted soon.

by Dan Goodin (US) - Jul 13, 2015 9:11am PDT

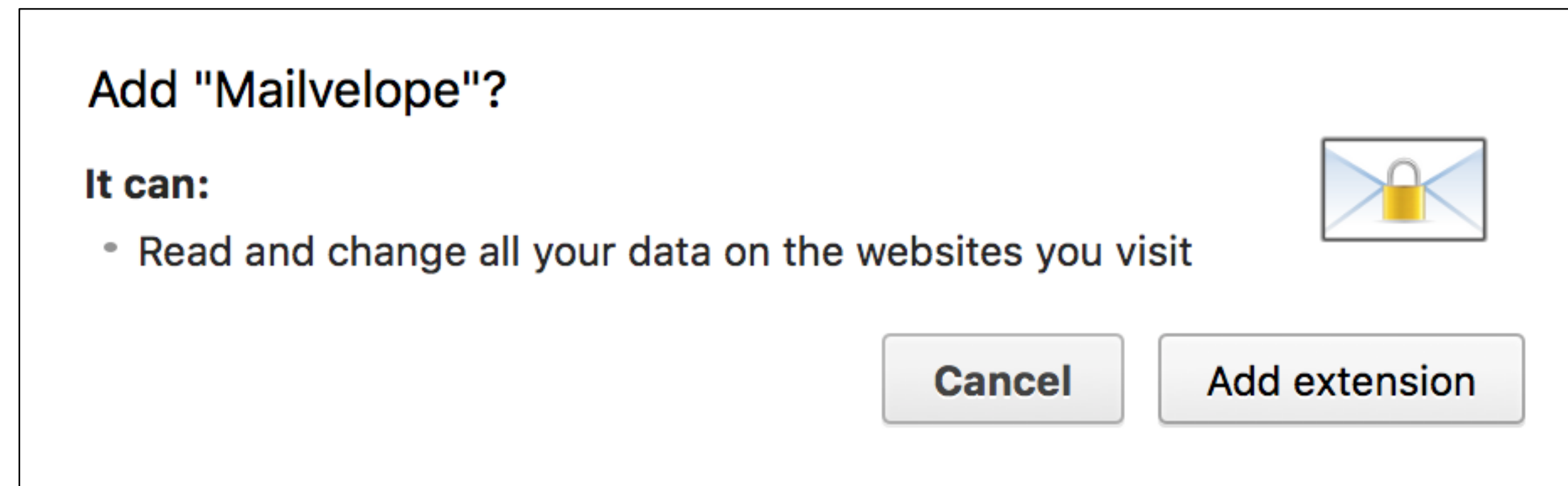
- **Good news:** plugin sandboxing improving, and need for plugins decreasing (due to HTML5 and extensions)

What about Browser Extensions?

- Most things you use today are probably extensions
- **Examples:** AdBlock, Ghostery, Mailvelope
 - Goal: Extend the functionality of the browser
- (Chrome:) Carefully designed security model to **protect from malicious websites**
 - **Privilege separation:** extensions consist of multiple components with well-defined communication
 - **Least privilege:** extensions request permissions

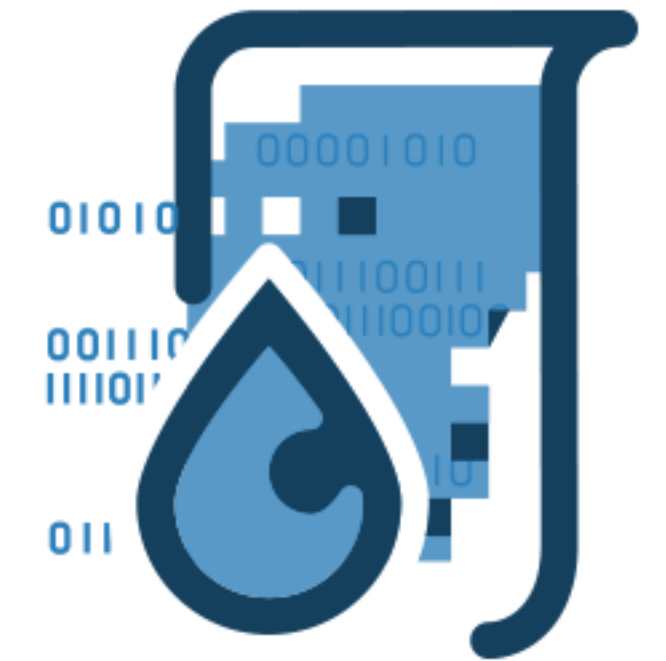
What about Browser Extensions?

- But be wary of malicious extensions: **not subject to the same-origin policy** – can inject code into any webpage!



Foreshadowing:

SOP Does Not Control Sending



- A webpage can **send** information to any site
- Can use this to send out secrets...Example: leak info via image

```

```